30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 8

Jetpack Compose Fundamentals — State, Recomposition, Side Effects & Stability



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 8

Why Compose is Day 8 in 2026

Compose is the default UI toolkit for new Android screens, and interviews have moved past "what is declarative UI". They probe **how Compose decides what to re-run**:

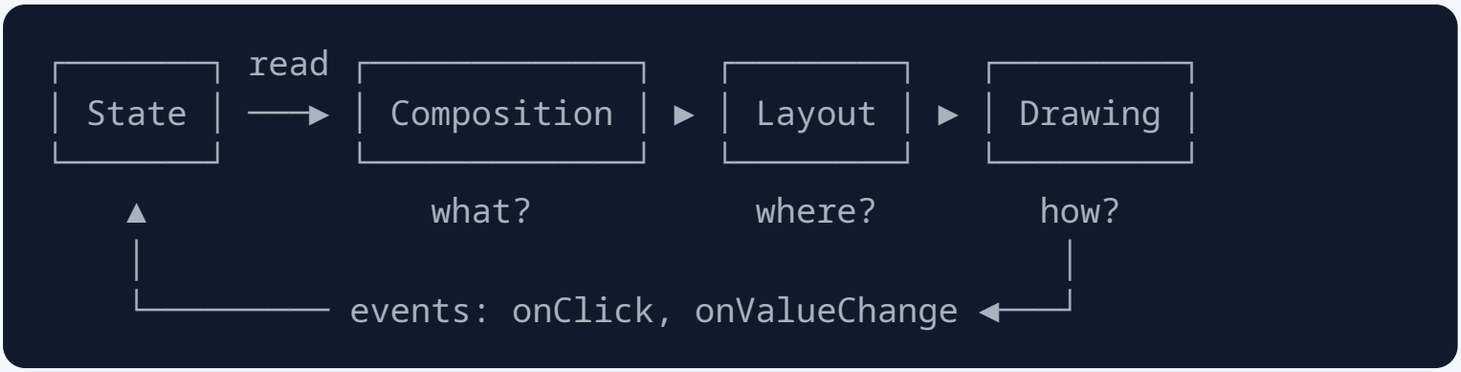
- Recomposition, skipping and **where** you read state
- `remember`, `rememberSaveable` and which state survives what
- Side-effect APIs — and the bugs you get when you misuse them
- Stability and **strong skipping mode**, on by default since Kotlin 2.0.20
- Theming, design systems and the same UI on iOS with Compose Multiplatform

Level	Questions
Junior	Recomposition · State & remember · State hoisting
Mid	Side effects · Modifiers · Lazy lists & keys · derivedStateOf
Senior	Stability & strong skipping · Phases & performance · CompositionLocal
Architect	Material 3 theming · Design-system APIs · Compose at scale



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: UI = f(state)



- A composable **describes** UI for the current state; it doesn't mutate views
- Writing a state object schedules work only for the scopes that **read** it
- Where you read state decides which phase re-runs: composition, layout or only drawing

THE QUESTION BEHIND EVERY COMPOSE ANSWER

"Who reads this state, and in which phase?" — Recomposition bugs, performance issues and effect bugs all come back to it.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 8

Junior

Recomposition · State & remember · State hoisting



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Composition and recomposition

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(text = "Hello, $name", modifier = modifier)
}

@Composable
fun Counter() {
    var count by remember { mutableIntStateOf(0) }
    Button(onClick = { count++ }) {
        Text("Clicked $count times")
    }
}
```

- **Composition:** the first run builds the UI tree and stores remembered values in the *slot table*
- **Recomposition:** when `count` changes, Compose re-runs only the composables and lambdas that **read** it
- **Skipping:** a composable whose inputs haven't changed is not re-run at all



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What interviewers check

INTERVIEW TRAP

"Recomposition redraws the whole screen." — **No.** Compose re-runs the smallest scopes that read the changed state and skips everything whose inputs are equal.

INTERVIEW TRAP

"A composable runs once, top to bottom." — It can be skipped, re-run on every animation frame, or run in any order, and a recomposition can be **cancelled** and thrown away. So the body must be fast, idempotent and free of side effects.

FOLLOW-UP

"Why does `remember` inside a `for` loop mix up state when items move?" — `remember` is **positional**: it's keyed by its place in the call tree. Wrap each item in `key(item.id) { … }` so the state follows the item.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. State: what survives what

API	Survives	Use for
`remember { mutableStateOf() }`	Recomposition	UI element state: expanded, selected
`rememberSaveable { … }`	+ configuration change **and** process death	Small user input: text, current tab
ViewModel `StateFlow`	+ configuration change (**not** process death)	Screen state and business logic
Repository / disk	Everything	The data itself

```
var query by rememberSaveable { mutableStateOf("") }
var expanded by remember { mutableStateOf(false) }
val selected = remember { mutableStateListOf<String>() }
val state by viewModel.uiState.collectAsStateWithLifecycle()
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. State traps

INTERVIEW TRAP

`val count = mutableStateOf(0)` **without** `remember` — a new state object is created on every recomposition, so the value keeps resetting. Lint flags it as `UnrememberedMutableState`.

INTERVIEW TRAP

`mutableStateOf(mutableListOf<Item>())` and then `.add(item)` — Compose only sees **writes to the state object**, not changes inside a plain list. Assign a new immutable list, or use `mutableStateListOf`.

INTERVIEW TRAP

Putting a large list in `rememberSaveable` — it goes into the saved-state `Bundle`, which is small and can throw `TransactionTooLargeException`. Save IDs; reload the data. Custom types need `@Parcelize` or a `Saver`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. State hoisting: stateless composables

```
@Composable
fun SearchBar(
    query: String,
    onQueryChange: (String) -> Unit,
    modifier: Modifier = Modifier,
) {
    OutlinedTextField(
        value = query,
        onValueChange = onQueryChange,
        modifier = modifier.fillMaxWidth(),
        singleLine = true,
    )
}
```

- Hoist state to the **lowest common ancestor** that reads or writes it: value down, event lambda up
- Stateless composables are trivial to preview, test and reuse
- `modifier: Modifier = Modifier` is the **first optional parameter**, applied once to the root



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Where the state should live

INTERVIEW TRAP

Passing the `ViewModel` down into child composables. It couples every child to one screen and breaks previews. Children receive **state and lambdas**; only the screen-level route touches the ViewModel (Day 3).

INTERVIEW TRAP

"Hoist everything into the ViewModel." — Pure UI element state (scroll position, an expanded card, focus) can stay in composition or in a **plain state holder** class. Business state goes to the ViewModel.

FOLLOW-UP

"What is a plain state holder?" — A class that groups UI logic and state, created with a `rememberXState()` function — like `LazyListState` or `DrawerState`. Your own `rememberSearchState()` follows the same pattern.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 8

→

Mid-Level

Side effects · Modifiers · Lazy lists & keys · derivedStateOf



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Side-effect APIs

API	Lifetime	Use for
`LaunchedEffect(key)`	Coroutine starts on enter, restarts when the key changes, cancelled on leave	Suspend work tied to UI: show a snackbar, run an animation
`rememberCoroutineScope()`	Scope cancelled when the call site leaves composition	Launch from callbacks: `onClick` → scroll, open drawer
`DisposableEffect(key)`	Setup + mandatory `onDispose`	Register and unregister listeners
`SideEffect`	After every successful recomposition	Publish Compose state to non-Compose code
`rememberUpdatedState(x)`	Latest value, effect not restarted	Long-running effect calling a changing lambda
`produceState`	Coroutine producing a `State`	Callback or suspend source → `State`

INTERVIEW TRAP

"With `rememberCoroutineScope` you must cancel the coroutines yourself." — **Wrong** (and v1 said it). The scope is cancelled automatically when the composable leaves the composition.



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. DisposableEffect in practice

```
@Composable
fun ScreenAnalytics(onStart: () -> Unit) {
    val lifecycleOwner = LocalLifecycleOwner.current
    val currentOnStart by rememberUpdatedState(onStart)

    DisposableEffect(lifecycleOwner) {
        val observer = LifecycleEventObserver { _, event ->
            if (event == Lifecycle.Event.ON_START) {
                currentOnStart()
            }
        }
        lifecycleOwner.lifecycle.addObserver(observer)
        onDispose {
            lifecycleOwner.lifecycle.removeObserver(observer)
        }
    }
}
```

- The key is the **lifecycle owner**: a new owner → dispose and set up again
- `rememberUpdatedState` gives the effect the newest lambda without restarting it
- Shortcut in `lifecycle-runtime-compose`: `LifecycleStartEffect` / `LifecycleResumeEffect`

INTERVIEW TRAP

Loading data in `LaunchedEffect(Unit)`. It re-runs whenever the screen re-enters composition — for example after every rotation. Loading belongs in the ViewModel (Day 3).



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Modifiers: order is meaning

```
Text(
    text = "Pay",
    modifier = Modifier
        .padding(16.dp)          // outer margin, not clickable
        .clip(RoundedCornerShape(12.dp))
        .background(MaterialTheme.colorScheme.primary)
        .clickable(onClick = onPay)
        .padding(horizontal = 24.dp, vertical = 12.dp),
)
```

- Each modifier **wraps** everything after it; the chain applies from the outside in
- Order changes size, touch area, clipping and drawing: swap `clip` and `background` and the corners are gone
- Modifier chains are immutable values — cheap to build, safe to reuse

INTERVIEW TRAP

Writing custom modifiers with `Modifier.composed { }`. It's no longer recommended for performance reasons — implement `Modifier.Node` with a `ModifierNodeElement`, or chain existing modifiers in a plain extension function.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Lazy lists: keys and content types

```
LazyColumn(state = listState) {
    items(
        items = messages,
        key = { it.id },
        contentType = { it.kind },
    ) { message ->
        MessageRow(message, Modifier.animateItem())
    }
}
```

- Only visible (and prefetched) items are composed
- `key`: item state and scroll position follow the **item**, not its index — without it, the key is the position
- `contentType`: items of the same type can reuse each other's composition
- `Modifier.animateItem()` animates insertions, removals and moves — it needs keys



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Lazy-list traps

INTERVIEW TRAP

A `LazyColumn` inside a `Column` with `verticalScroll` crashes: a lazy list can't be measured with infinite height. Use **one** lazy list with several `item { }` / `items { }` sections.

INTERVIEW TRAP

Keys must be **unique** (a duplicate key crashes) and, on Android, a type a `Bundle` can store — `String`, `Int`, `Long`. Don't use the whole data object as a key.

INTERVIEW TRAP

Items with zero size before their data loads (e.g. empty images). The list thinks they all fit on screen and composes far too many. Give placeholders a size.

FOLLOW-UP

"Column or LazyColumn for 20 fixed rows?" — `Column` composes and lays out all children up front; fine for small, bounded content. Unbounded or long data → lazy.



Q7. derivedStateOf or remember(key)?

```
val listState = rememberLazyListState()

// Recomposes whenever the first visible index changes
val showBad = listState.firstVisibleItemIndex > 0

// Recomposes only when the Boolean flips
val showScrollToTop by remember {
    derivedStateOf { listState.firstVisibleItemIndex > 0 }
}
```

Tool	Inputs	Recalculates
`remember(a, b) { f(a, b) }`	Parameters or plain values	When a key changes
`derivedStateOf { }`	State objects that change **more often** than the result	When the result changes

INTERVIEW TRAP

`remember { derivedStateOf { count > 0 } }` where `count` is a plain **parameter**, not state — it's never tracked, so the value is stale forever. Use `remember(count) { … }` or compute it directly.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 8



Senior

Stability & strong skipping · Phases & performance · CompositionLocal



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Stability and strong skipping mode

KOTLIN 2.0.20+

Strong skipping mode is **on by default**. The Compose compiler now ships with Kotlin — plugin `org.jetbrains.kotlin.plugin.compose`, same version as Kotlin.

	Before strong skipping	Strong skipping (default)
Composable with an unstable parameter	Never skipped	Skippable
Unstable parameters compared by	—	Instance equality (`===`)
Stable parameters compared by	`equals()`	`equals()`
Lambdas inside composables	Remembered only if all captures are stable	Always remembered; captures act as keys

Stable: primitives, `String`, function types, enums, classes whose public properties are all `val` of stable types, `@Immutable` / `@Stable` types, kotlinx immutable collections.



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. What still goes wrong

```
// Unstable: a `var`, and List is an interface
data class CartUi(var items: List<Item>)

// Stable (if Item is stable): val + immutable collection
data class CartUi(val items: ImmutableList<Item>)
```

```
composeCompiler {
    reportsDestination = layout.buildDirectory.dir("compose")
    metricsDestination = layout.buildDirectory.dir("compose")
    stabilityConfigurationFiles.add(
        rootProject.layout.projectDirectory
            .file("stability.conf")
    )
}
```

INTERVIEW TRAP

"Data classes are stable." — v1 said it. Only if every property is a `val` of a stable type. A `var`, a `List`, or a class from a module **without** the Compose compiler makes it unstable.

INTERVIEW TRAP

"Strong skipping made stability irrelevant." — Unstable parameters are compared with `===`. A new but equal `List` (every `map {}` in the ViewModel) still recomposes. And lambdas no longer need manual `remember` — v1 advice is outdated.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. The three phases — defer state reads

```
Composition ──▶ Layout (measure, place) ──▶ Drawing
 reads here:     reads here:                 reads here:
 re-run all 3    re-run layout + draw        re-run draw
```

```kotlin
val color by animateColorAsState(target, label = "bg")
val fade by animateFloatAsState(targetAlpha, label = "a")

// Read in composition: recomposes on every animation frame
Box(Modifier.background(color))

// Read in the draw phase: no recomposition
Box(Modifier.drawBehind { drawRect(color) })

// Lambda versions defer the read to layout / draw
Box(Modifier.offset { IntOffset(0, scroll.value / 2) })
Box(Modifier.graphicsLayer { alpha = fade })
```

INTERVIEW TRAP

"`graphicsLayer` *never triggers recomposition.*" — Only the **lambda** version, reading state inside the block. `Modifier.graphicsLayer(alpha = fade)` reads `fade` in composition, so it recomposes every frame.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Performance basics checklist

- Measure a **release** build with R8 and Baseline Profiles (Day 1) — debug Compose is much slower and misleading
- **Layout Inspector** shows recomposition and skip counts per composable
- **Compose compiler reports**: which composables are skippable and which parameters are unstable
- Heavy work (sorting, formatting, parsing) out of composition — ViewModel or `remember(keys)`
- Trace real jank with system tracing / Perfetto — the deep dive is Day 17

INTERVIEW TRAP

Backwards writes: writing a state object after reading it in the same composition — e.g. `count++` directly in the body. It schedules another recomposition, which writes again: an endless loop.

INTERVIEW TRAP

"Add `@Stable` everywhere to be safe." — It's a **promise** to the compiler, not a check. Lie about a mutable class and Compose skips updates: stale UI that's hard to debug.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. CompositionLocal: implicit, tree-scoped values

```
@Immutable
data class Spacing(
    val small: Dp = 4.dp,
    val medium: Dp = 8.dp,
    val large: Dp = 16.dp,
)

val LocalSpacing = staticCompositionLocalOf { Spacing() }

val roomy = Spacing(large = 24.dp)
CompositionLocalProvider(LocalSpacing provides roomy) {
    Content() // reads LocalSpacing.current.large
}
```

	`compositionLocalOf`	`staticCompositionLocalOf`
Value changes	Recompose only the **readers**	Recompose the **whole** provided content
Best for	Values that change often	Values that rarely or never change (theme tokens)



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. When not to use it

- Good fits: theme tokens, `LocalContext`, `LocalDensity`, `LocalLayoutDirection` — tree-scoped, with a sensible default
- Every read is an **implicit dependency**: invisible in the signature, easy to forget in previews and tests

INTERVIEW TRAP

Using CompositionLocal to pass ViewModels, repositories or screen data "to avoid parameters". It hides dependencies and breaks previews. Pass explicit parameters; use DI for services (Day 7).

INTERVIEW TRAP

`LocalContext.current as Activity` — the context can be a `ContextWrapper` (dialogs, themed wrappers), so the cast can crash. Unwrap the `baseContext` chain, or better, don't need the Activity in UI code.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 8

→

Architect

Material 3 theming · Design-system APIs · Compose at scale



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Material 3 theming

```
@Composable
fun AppTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit,
) {
    val context = LocalContext.current
    val colors = when {
        dynamicColor && Build.VERSION.SDK_INT >= 31 ->
            if (darkTheme) dynamicDarkColorScheme(context)
            else dynamicLightColorScheme(context)
        darkTheme -> DarkColors
        else -> LightColors
    }
    MaterialTheme(colorScheme = colors, typography = AppTypo) {
        CompositionLocalProvider(
            LocalSpacing provides Spacing(),
            content = content,
        )
    }
}
```

- Components use color **roles** (`colorScheme.primary`, `onSurface`), the type scale and shapes — never hex values
- Dynamic color from Android 12 (API 31); brand colors as the fallback
- Tokens Material doesn't have (spacing, elevation levels) → your own CompositionLocals



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Theming traps

INTERVIEW TRAP

Hard-coded `Color(0xFF…)` inside components. Dark theme, dynamic color and high-contrast themes silently break. Only the theme file knows hex values.

INTERVIEW TRAP

Mixing Material 2 (`androidx.compose.material`) and Material 3 components. M2 components read the **M2** `MaterialTheme`, so inside an M3 theme they fall back to default colors and typography.

INTERVIEW TRAP

Ignoring window insets. With targetSdk 35+, edge-to-edge is enforced: content draws behind the system bars. Use the `innerPadding` from `Scaffold`, or `Modifier.safeDrawingPadding()`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Design-system component APIs

```
@Composable
fun AppCard(
    title: @Composable () -> Unit,
    modifier: Modifier = Modifier,
    actions: @Composable RowScope.() -> Unit = {},
    content: @Composable ColumnScope.() -> Unit,
) {
    val spacing = LocalSpacing.current
    Card(modifier) {
        Column(Modifier.padding(spacing.large)) {
            val style = MaterialTheme.typography.titleMedium
            ProvideTextStyle(style) { title() }
            content()
            Row(Modifier.align(Alignment.End)) { actions() }
        }
    }
}
```

- Parameter order from the Compose API guidelines: required → `modifier` → optional → trailing `content`
- **Slots** (`@Composable` lambdas) instead of strings and flags: callers decide what goes inside
- Stateless by default; previews for every state; screenshot tests in CI (Day 10)



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Running a component library

INTERVIEW TRAP

Boolean explosion: `AppButton(isPrimary, isOutlined, hasIcon, isLoading, …)`. Every flag doubles the states to test. Split it into `PrimaryButton` / `OutlinedButton` and use slots for icons.

INTERVIEW TRAP

"A shared design system across Android and iOS needs `expect` / `actual` for every component." — v1 advice. With **Compose Multiplatform**, components live in `commonMain`; `expect` / `actual` only for the rare platform detail.

GOVERNANCE

Custom Lint rules (or Slack's `compose-lints`) that forbid raw `material3.Button` in feature modules keep teams on the design system. Version the library with semver and publish migration notes.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Compose at scale: interop

```
@Composable
fun LegacyChart(
    points: List<Point>,
    modifier: Modifier = Modifier,
) {
    AndroidView(
        factory = { context -> ChartView(context) },
        modifier = modifier,
        update = { view -> view.setPoints(points) },
        onRelease = { view -> view.clear() },
    )
}
```

- `factory` runs once; `update` runs again whenever the state it reads changes
- In lazy lists, pass `onReset` so the View can be **reused** instead of recreated
- Compose inside Fragments (`ComposeView`, `ViewCompositionStrategy`) and Fragments inside Compose (`AndroidFragment`) were Day 2
- Migrate bottom-up: theme first, then leaf components, then whole screens

INTERVIEW TRAP

Treating `update` as one-time setup. It re-runs whenever a state it reads changes — keep it cheap and idempotent; create listeners in `factory`, not in `update`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Performance guardrails in CI

Guardrail	Catches
Compose compiler reports / metrics diffed in CI	New unstable parameters, non-skippable composables
Stability configuration file	Domain types from non-Compose modules treated as unstable
Baseline Profiles (Day 1, Day 18)	Slow first frames and first scrolls
Macrobenchmark `FrameTimingMetric`	Scroll and animation jank regressions
Lint / `compose-lints`	Missing `modifier` parameter, unremembered state, misuse of effects

SENIOR ANSWER IN ONE LINE

"Compose re-runs whatever reads the state that changed. Keep state small and hoisted, types stable, reads late, and side effects inside effect APIs — then measure in a release build."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 8 · ARCHITECT

Day 8 — cheat sheet

- UI = f(state). Recomposition re-runs only the scopes that **read** changed state; unchanged inputs are skipped.
- Composable bodies: fast, idempotent, no side effects — they can run often, in any order, or be cancelled.
- `remember` survives recomposition; `rememberSaveable` also rotation and process death; the ViewModel only rotation.
- Hoist state: value down, events up. `modifier` is the first optional parameter.
- Effects: `LaunchedEffect` (keys!), `DisposableEffect` + `onDispose`, `rememberCoroutineScope` for callbacks — auto-cancelled.
- Modifier order is meaning. New custom modifiers use `Modifier.Node`, not `composed`.
- Lazy lists: stable unique `key`, `contentType`, never nest same-direction scrolling.
- Strong skipping is default (Kotlin 2.0.20+): unstable params compare by `===`; data classes aren't automatically stable.
- Defer reads with lambda modifiers (`offset {}`, `graphicsLayer {}`, `drawBehind {}`); measure in release builds.
- Theme with M3 color roles and CompositionLocal tokens — never hex values in components.

Day 9 is next → Testing, part 1. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700